

Biconoirs Gourmet - Backend Business Rules

Postman testing guide for all current URIs of the NestJS + Prisma + Supabase project.

Base Configuration

Recommended Postman variable:

```
baseUrl = http://localhost:3001
```

When deploying on AWS:

```
baseUrl = http://PUBLIC_IP_EC2:3001
```

Backend A base (consumed internally):

```
http://3.20.57.154:3000/ops
```

Common headers:

```
Content-Type: application/json
```

```
Key: x-user-id
```

```
Value: usr_002
```

Suggested variables in Postman:

```
baseUrl
userId
adminUserId
customerId
dishId
categoryId
orderId
inventoryId
skuCode
refreshToken
resetToken
```

Ingredients

GET /ops/ingredients/:ingredientId

Get a single ingredient by its ID.

Method	GET
URL	{{baseUrl}}/ops/ingredients/{{ingredientId}}

Body:

Without body.

POST /ops/ingredients

Create a new ingredient.

Method	POST
URL	{{baseUrl}}/ops/ingredients

Body JSON:

```
{
  "name": "Tomato",
  "category": "Vegetables",
  "description": "Fresh red tomato",
  "unit": "kg",
  "unit_cost": 1.5
}
```

PUT /ops/ingredients/:ingredientId

Full update of an ingredient.

Method	PUT
URL	{{baseUrl}}/ops/ingredients/{{ingredientId}}

Body JSON:

```
{
  "name": "Tomato Updated",
  "category": "Vegetables",
  "unit": "kg",
  "unit_cost": 1.8
}
```

PATCH /ops/ingredients/:ingredientId

Partial update of an ingredient.

Method	PATCH
URL	{{baseUrl}}/ops/ingredients/{{ingredientId}}

Body JSON:

```
{
  "unit_cost": 2.0
}
```

DELETE /ops/ingredients/:ingredientId

Remove an ingredient by its ID.

Method	DELETE
URL	{{baseUrl}}/ops/ingredients/{{ingredientId}}

Body:

Without body.

Surveys

GET /ops/surveys

List all surveys.

Method	GET
URL	{{baseUrl}}/ops/surveys

Body:

Without body.

GET /ops/surveys/:surveyId

Get details of a specific survey including its responses.

Method	GET
URL	{{baseUrl}}/ops/surveys/{{surveyId}}

Body:

Without body.

POST /ops/surveys/:surveyId/submit

Submit responses to a survey.

Method	POST
URL	{{baseUrl}}/ops/surveys/{{surveyId}}/submit

Body JSON:

```
{
  "user_id": "{{userId}}",
```

```
"responses": [
  {
    "question_id": "q-001",
    "answer": "Very good"
  },
  {
    "question_id": "q-002",
    "answer": "5"
  }
]
```

Auth

POST /api/v1/auth/signup

Register a new user. Forwards the request to Backend A.

Method	POST
URL	{{baseUrl}}/api/v1/auth/signup

Body JSON:

```
{
  "name": "Juan Perez",
  "email": "juan@test.com",
  "password": "clave12345",
  "phone": "0999999999"
}
```

POST /api/v1/auth/login

Authenticate user by email and password.

Method	POST
URL	{{baseUrl}}/api/v1/auth/login

Body JSON:

```
{
  "email": "juan@test.com",
  "password": "clave12345"
}
```

POST /api/v1/auth/logout

Revoke the user session token.

Method	POST
---------------	------

URL	{{baseUrl}}/api/v1/auth/logout
------------	--------------------------------

Headers:

authorization: {{refreshToken}}

Body:

Without body.

POST /api/v1/auth/password-recovery

Send a password recovery email.

Method	POST
URL	{{baseUrl}}/api/v1/auth/password-recovery

Body JSON:

<pre>{ "email": "juan@test.com" }</pre>

Profile

GET /api/v1/profile

Get the authenticated user's profile data.

Method	GET
URL	{{baseUrl}}/api/v1/profile

Headers:

x-user-id: {{userId}}

Body:

Without body.

PUT /api/v1/profile

Full update of the user profile.

Method	PUT
---------------	-----

URL	{{baseUrl}}/api/v1/profile
------------	----------------------------

Headers:

x-user-id: {{userId}}

Body JSON:

<pre>{ "name": "Juan Perez Updated", "phone": "0988888888", "address": "Av. Principal 123", "preferences": { "favoriteCategory": "desserts", "notifications": true } }</pre>
--

PATCH /api/v1/profile

Partial update of the user profile.

Method	PATCH
URL	{{baseUrl}}/api/v1/profile

Headers:

x-user-id: {{userId}}

Body JSON:

<pre>{ "phone": "0977777777" }</pre>
--

Menu

GET /api/v1/menu

List available dishes. Supports optional query filters.

Method	GET
URL	{{baseUrl}}/api/v1/menu

Optional query params:

?category=Pastas

?search=lasagna

?available=true
?page=1&perPage=10

Body:

Without body.

GET /api/v1/menu/:dishId

Get details of a specific dish.

Method	GET
URL	{{baseUrl}}/api/v1/menu/{{dishId}}

Body:

Without body.

Cart

POST /api/v1/cart/add

Add a dish to the user's cart.

Method	POST
URL	{{baseUrl}}/api/v1/cart/add

Headers:

x-user-id: {{userId}}

Body JSON:

```
{
  "dish_id": "{{dishId}}",
  "quantity": 2
}
```

GET /api/v1/cart

View the current user's cart with total.

Method	GET
URL	{{baseUrl}}/api/v1/cart

Headers:

x-user-id: {{userId}}

Body:

Without body.

Checkout

POST /api/v1/checkout

Process the cart as an order. Clears the cart after success.

Method	POST
URL	{{baseUrl}}/api/v1/checkout

Headers:

x-user-id: {{userId}}

Body JSON:

```
{
  "delivery_type": "delivery",
  "delivery_address": "Av. Principal 123",
  "special_instructions": "No onion"
}
```

Orders

GET /api/v1/orders

List the authenticated user's orders. Supports optional filters.

Method	GET
URL	{{baseUrl}}/api/v1/orders

Headers:

x-user-id: {{userId}}

Optional query params:

?status=pending

?from=2026-01-01&to=2026-12-31

Body:

Without body.

Reservations

POST /api/v1/reservations

Create a new table reservation.

Method	POST
URL	{{baseUrl}}/api/v1/reservations

Headers:

x-user-id: {{userId}}

Body JSON:

```
{
  "reservation_date": "2026-07-20",
  "reservation_time": "19:30",
  "party_size": 4,
  "special_requests": "Window table preferred"
}
```

GET /api/v1/reservations

List the authenticated user's reservations.

Method	GET
URL	{{baseUrl}}/api/v1/reservations

Headers:

x-user-id: {{userId}}

Body:

Without body.

PUT /api/v1/reservations/:reservationId

Full update of a reservation.

Method	PUT
URL	{{baseUrl}}/api/v1/reservations/{{reservationId}}

Headers:

```
x-user-id: {{userId}}
```

Body JSON:

```
{
  "reservation_time": "20:00",
  "party_size": 6
}
```

PATCH /api/v1/reservations/:reservationId

Partial update of a reservation.

Method	PATCH
URL	{{baseUrl}}/api/v1/reservations/{{reservationId}}

Headers:

```
x-user-id: {{userId}}
```

Body JSON:

```
{
  "party_size": 2
}
```

DELETE /api/v1/reservations/:reservationId

Cancel and delete a reservation.

Method	DELETE
URL	{{baseUrl}}/api/v1/reservations/{{reservationId}}

Headers:

```
x-user-id: {{userId}}
```

Body:

Without body.

Surveys

GET /api/v1/surveys/active

List all currently active surveys.

Method	GET
URL	{{baseUrl}}/api/v1/surveys/active

Body:

Without body.

POST /api/v1/surveys/:surveyId/respond

Submit the authenticated user's responses to a survey.

Method	POST
URL	{{baseUrl}}/api/v1/surveys/{{surveyId}}/respond

Headers:

x-user-id: {{userId}}

Body JSON:

```
{
  "responses": [
    {
      "question_id": "q-001",
      "answer": "Excellent"
    },
    {
      "question_id": "q-002",
      "answer": "5"
    }
  ]
}
```

Admin

GET /api/v1/admin/inventory

List all inventory items. Requires admin role.

Method	GET
URL	{{baseUrl}}/api/v1/admin/inventory

Headers:

x-user-id: {{adminUserId}}

Body:

Without body.

POST /api/v1/admin/inventory

Create a new inventory item with its associated ingredient. Requires admin role.

Method	POST
URL	{{baseUrl}}/api/v1/admin/inventory

Headers:

x-user-id: {{adminUserId}}

Body JSON:

```
{
  "ingredient_name": "Tomato",
  "current_stock": 25,
  "unit": "kg",
  "reorder_level": 5,
  "supplier": "Local Supplier",
  "unit_cost": 1.5,
  "expiry_date": "2026-07-15"
}
```

PUT /api/v1/admin/inventory/:inventoryId

Full update of an inventory item. Requires admin role.

Method	PUT
URL	{{baseUrl}}/api/v1/admin/inventory/{{inventoryId}}

Headers:

x-user-id: {{adminUserId}}

Body JSON:

```
{
  "current_stock": 40,
  "reorder_level": 10,
  "expiry_date": "2026-08-20"
}
```

PATCH /api/v1/admin/inventory/:inventoryId

Partial update of an inventory item. Requires admin role.

Method	PATCH
URL	{{baseUrl}}/api/v1/admin/inventory/{{inventoryId}}

Headers:

x-user-id: {{adminUserId}}

Body JSON:

```
{
  "current_stock": 15
}
```

DELETE /api/v1/admin/inventory/:inventoryId

Remove an inventory item. Requires admin role.

Method	DELETE
URL	{{baseUrl}}/api/v1/admin/inventory/{{inventoryId}}

Headers:

x-user-id: {{adminUserId}}

Body:

Without body.

GET /api/v1/admin/analytics

Get analytics: total orders, revenue, reservations, and low stock items. Requires admin role.

Method	GET
URL	{{baseUrl}}/api/v1/admin/analytics

Headers:

x-user-id: {{adminUserId}}

Optional query params:

?from=2026-01-01&to=2026-12-31

Body:

Without body.

Recommended Test Order

1. POST /api/v1/auth/signup
2. POST /api/v1/auth/login — copy userId from response
3. GET /api/v1/menu — copy a real dishId
4. POST /api/v1/cart/add with x-user-id
5. GET /api/v1/cart
6. POST /api/v1/checkout
7. GET /api/v1/orders
8. POST /api/v1/reservations
9. GET /api/v1/reservations
10. GET /api/v1/surveys/active — copy a real surveyId
11. POST /api/v1/surveys/{{surveyId}}/respond
12. GET /ops/ingredients — (Backend A)
13. GET /ops/surveys — (Backend A)
14. POST /api/v1/admin/inventory with adminUserId
15. GET /api/v1/admin/analytics

Important Notes

If a protected route returns 401, it is usually because the x-user-id header is missing or the user does not exist in the database.

If an admin endpoint responds 403, the user exists but does not have the admin role.

If it returns 404, check that the ID exists in the database and that the URL path is correct.

The Layer B backend (yours) forwards requests internally to Backend A at <http://3.20.57.154:3000/ops>. You never call Backend A directly from Postman.

For AWS deployment, replace baseUrl with the configured public IP or domain of your Backend B instance.